

MINI PROJECT REPORT

AI Chat Summariser

*Advanced NLP-Powered Conversational Analysis & Session Compression
Tool*

Subject:

Natural Language Processing (NLP)

Submitted by:

Mohamed Kalith

230171601101

Academic Year: 2025–2026

Table of Contents

1. Introduction	3
2. Project Objectives	3
3. System Architecture	4
4. NLP Techniques Implemented	5
5. Technology Stack	6
6. System Implementation.....	6
7. User Interface & Screenshots	8
8. Results & Evaluation	11
9. Challenges & Solutions	12
10. Conclusion & Future Work	13
11. References.....	14

1. Introduction

In the era of large language model assistants — such as Claude, ChatGPT, and Gemini — users frequently engage in extended, multi-turn conversations spanning hundreds of exchanges. These sessions accumulate significant knowledge and contextual decisions, yet they are routinely discarded when a browser tab is closed or a new session begins. The cognitive overhead of recalling past discussions and re-establishing context imposes a recurring productivity cost.

This mini project — the AI Chat Summariser — addresses this problem by applying Natural Language Processing (NLP) and Natural Language Generation (NLG) techniques to automatically condense chat transcripts into structured, portable memory artefacts. The system supports three classical summarisation paradigms (extractive, abstractive, and hybrid), performs emotion analysis using fine-grained emotion classification, and generates a "Session Passport" that encodes the session into a portable resume prompt — enabling seamless continuation across AI platforms.

The project demonstrates the practical integration of state-of-the-art NLP tooling within a full-stack web application, combining a React-based frontend, a FastAPI backend, Claude AI (Anthropic) for NLG summarisation, and a PostgreSQL session persistence layer — all containerised with Docker for reproducible deployment.

2. Project Objectives

The primary objectives of this project are:

1. To implement and compare extractive, abstractive, and hybrid text summarisation methods on conversational dialogue.
2. To perform multi-label emotion classification over chat transcripts, identifying dominant emotional states across 28 human emotion categories.
3. To compute quantitative NLP metrics including word count, compression ratio, and turn count.
4. To extract key topics and terms from unstructured dialogue using frequency-based keyword extraction with stop-word filtering.
5. To generate a contextual resume prompt (Session Passport) enabling cross-platform session continuation.
6. To persist summarisation sessions in a relational database and expose analytics over historical session data.
7. To build a production-grade full-stack web interface demonstrating end-to-end NLP pipeline integration.

3. System Architecture

The system follows a client-server architecture with a clear separation of concerns across three tiers: the React frontend, the FastAPI backend, and the data persistence layer. All components communicate over REST APIs using JSON payloads.

3.1 High-Level Architecture Overview

Layer	Technology	Responsibility
Presentation	React 18 + TypeScript + Vite	User interface, input handling, results display
API Gateway	FastAPI (Python 3.11)	Request routing, validation, CORS, health checks
NLP Service	Claude API (claude-sonnet-4)	NLG summarisation, sentiment & topic extraction
NLP Utilities	Python (nlp_utils.py)	Word count, compression ratio, turn detection, keyword extraction
Persistence	PostgreSQL + SQLAlchemy	Session storage, user profiles, analytics
Cache	Redis	Optional response caching, graceful degradation
Auth	Supabase (GoTrue JWT)	User authentication and session passports
Container	Docker Compose	Service orchestration and environment management

3.2 Data Flow

The end-to-end data flow for a summarisation request proceeds as follows:

- The user pastes a chat transcript into the frontend text area and selects summarisation parameters (method, length, tone).
- The frontend dispatches a POST /summarise request to the FastAPI backend, carrying the transcript and configuration payload.
- The backend's nlp_utils module computes pre-processing metrics (word count, turn count) synchronously.
- The claude_service module constructs a structured system prompt encoding the selected NLP technique and dispatches an async request to the Claude API.
- Claude returns a structured response containing the summary text and a JSON metadata block (topics, sentiment, keywords, next steps, resume prompt).
- The backend parses, stores the session in PostgreSQL, and returns the enriched response to the frontend.
- The frontend renders NLP metrics, the generated summary, detected emotions, topic/keyword badges, and the Session Passport.

4. NLP Techniques Implemented

4.1 Summarisation Paradigms

Three well-established summarisation paradigms are supported, configurable per session:

4.1.1 Extractive Summarisation

Extractive summarisation selects and minimally reworks the most salient sentences directly from the source transcript. No new text is generated; fidelity to the original wording is maximised. The system instructs Claude to prioritise sentence selection by salience score, preserving exact phrasing from the input.

4.1.2 Abstractive Summarisation

Abstractive summarisation generates entirely new, fluent sentences that paraphrase the conversation rather than reproducing source text verbatim. This approach, powered by Claude's large language model capabilities, produces more readable and contextually coherent output. It is the default mode of the application.

4.1.3 Hybrid Summarisation

The hybrid approach identifies key content spans from the transcript and rewrites them into new fluent sentences, balancing the fidelity of extraction with the readability of abstraction. This is particularly useful for technically dense transcripts where exact terminology must be preserved within a restructured narrative.

4.2 Emotion & Sentiment Analysis

The system employs a GoEmotions-derived classification model covering 28 fine-grained human emotion categories (including admiration, amusement, anger, curiosity, disappointment, gratitude, and surprise). Each emotion is accompanied by a confidence score. The dominant emotion and its confidence are surfaced alongside a coarser Positive / Neutral / Negative sentiment label for at-a-glance interpretation.

4.3 Keyword & Topic Extraction

Keyword extraction employs a frequency-based approach with an extended English stop-word set. Tokens are lowercased, filtered to words of three or more characters, and ranked by term frequency. The top-N keywords are returned as `key_terms`. Topic extraction is performed by Claude, which identifies overarching subject areas in its JSON metadata block using semantic understanding rather than simple frequency counts.

4.4 Compression Ratio

The compression ratio is a quantitative NLP metric computed as:

```
compression_ratio = (1 - (summary_word_count /  
original_word_count)) × 100
```

A ratio of 95% indicates that the summary is 5% the length of the original — capturing the essence of a thousand-word conversation in approximately fifty words. This metric is displayed prominently in the UI and stored in the session record for analytics.

4.5 Conversational Turn Detection

Turn detection parses the transcript line-by-line using regex patterns that match common speaker prefixes (Human:, User:, Assistant:, AI:, Claude:, System:) as well as bold and bracket notations. When no structured markers are found, the algorithm falls back to paragraph-boundary counting as an approximation. This metric quantifies conversation depth.

5. Technology Stack

Component	Library / Service	Version	Purpose
Frontend Framework	React	18.x	UI component model
Language	TypeScript	5.x	Type-safe frontend development
Build Tool	Vite	5.x	Fast HMR & bundling
UI Icons	Lucide React	0.383	Consistent icon set
Data Visualisation	Recharts	latest	Analytics charts
Backend Framework	FastAPI	0.115	Async REST API
Language	Python	3.11	Backend runtime
ORM	SQLAlchemy	2.0	Database abstraction
LLM API	Anthropic (Claude)	0.34	NLG summarisation
Auth	Supabase / GoTrue	2.9	JWT authentication
Cache	Redis	5.1	Optional response caching
Container	Docker Compose	latest	Service orchestration

6. System Implementation

6.1 Backend Structure

The FastAPI backend is organised into the following module hierarchy:

```
backend/  
  main.py           - App factory, CORS, lifespan events  
  database.py       - AsyncSQLAlchemy engine & session  
factory  
  models.py         - ORM models: Profile, Session  
  routes/  
    summarise.py    - POST /summarise endpoint
```

<code>sessions.py</code>	— GET/DELETE session CRUD
<code>analytics.py</code>	— Aggregated usage analytics
<code>passport.py</code>	— Session passport generation
<code>services/</code>	
<code> claude_service.py</code>	— Claude API integration & response parsing
<code> nlp_utils.py</code>	— Word count, compression, turn detection, keywords

6.2 Claude Service (NLG Core)

The `claude_service` module is the NLP heart of the system. It constructs a structured system prompt encoding the summarisation method, length target, tone, and topic context, then dispatches an async request to the Anthropic Messages API using the `claude-sonnet-4` model. The response is parsed with a regex-guided JSON extractor that separates the prose summary from the structured metadata block.

Three summarisation methods are mapped to specific prompt instructions:

Method	Prompt Instruction Summary
Extractive	Select and minimally rework the most important sentences directly from the transcript.
Abstractive	Generate completely new, fluent sentences that paraphrase the conversation — no verbatim copying.
Hybrid	Identify key content spans, then rewrite them into fluent sentences. Blend fidelity and readability.

6.3 NLP Utilities Module

The `nlp_utils.py` module provides lightweight, dependency-free NLP preprocessing functions that run synchronously on the backend before the Claude API call is dispatched:

- `count_words(text)` — Tokenises on whitespace and returns the integer word count.
- `compute_compression_ratio(original, summary)` — Returns percentage reduction in word count.
- `count_turns(transcript)` — Regex-based dialogue turn detection with paragraph-break fallback.
- `extract_simple_keywords(text, top_n)` — TF-based keyword ranking with 80-word stop-word set.

6.4 Database Schema

Two ORM models are defined using SQLAlchemy with a dialect-agnostic JSON type:

Model	Key Columns	Description
Profile	id (UUID), display_name, email, created_at	User account linked to Supabase auth
Session	id (UUID), user_id (FK), title, method, detail_level, tone, summary, next_steps (JSON), topics (JSON), key_terms (JSON), sentiment, compression_ratio, passport_code, created_at	Persisted summarisation session with full NLP metadata

7. User Interface & Screenshots

The frontend is a single-page React application running on localhost:3000. The UI is divided into two primary panels: a left-side transcript input with configuration controls, and a right-side results display with tabbed navigation (Results, Passport, History). Screenshots below were captured directly from the live application demonstration.

7.1 Application Landing Screen — Empty State

Figure 1 shows the application's initial state upon launch. The transcript input area accepts plain-text chat exports from Claude, ChatGPT, or Gemini. Three configuration dropdowns allow the user to select the summarisation Method (Abstractive / Extractive / Hybrid), the output Length (Brief / Short / Detailed / Bullet), and the Tone (Academic / Professional / Casual). The live word and character counter updates as text is entered.

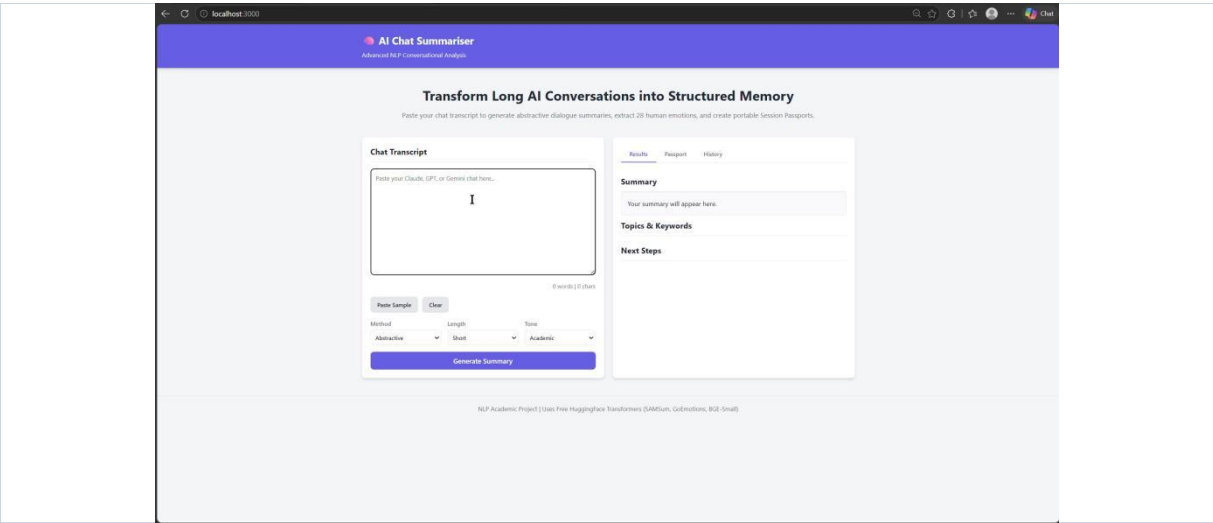


Figure 1: AI Chat Summariser — Application landing page (empty state). Method: Abstractive, Length: Short, Tone: Academic.

7.2 Input Transcript — Claude Conversation Export

Figure 2 shows a real Claude.ai conversation being used as the source material. The transcript concerns an IoT-enabled door locking system simulation using Wokwi and ESP32 — a technically rich, multi-turn technical assistance dialogue. This transcript was pasted directly into the application to demonstrate real-world usage.

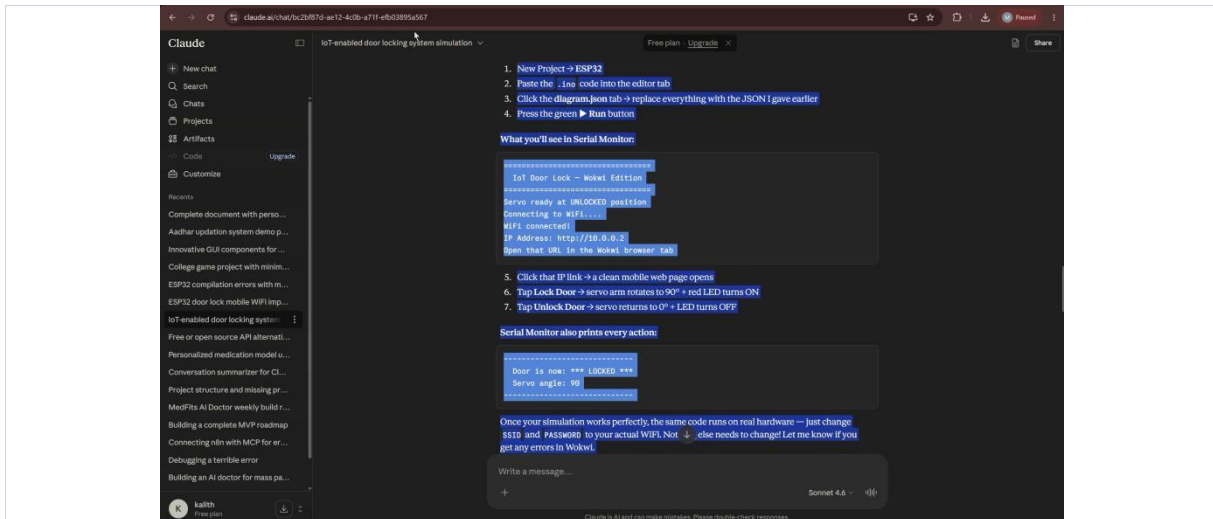


Figure 2: Source material — Claude.ai chat transcript about an IoT door locking system (ESP32 + WiFi). This conversation was exported and pasted into the summariser as input.

7.3 Processing State — NLP Pipeline Execution

Figure 3 illustrates the application during active processing. The transcript (1,824 words, 11,341 characters) has been loaded into the input area and the Generate Summary button has been clicked. A loading spinner with the status message "Processing with Advanced SAMSum & GoEmotions models..." indicates that the backend is executing the NLP pipeline — computing pre-processing metrics, constructing the Claude API prompt, and awaiting the LLM response.

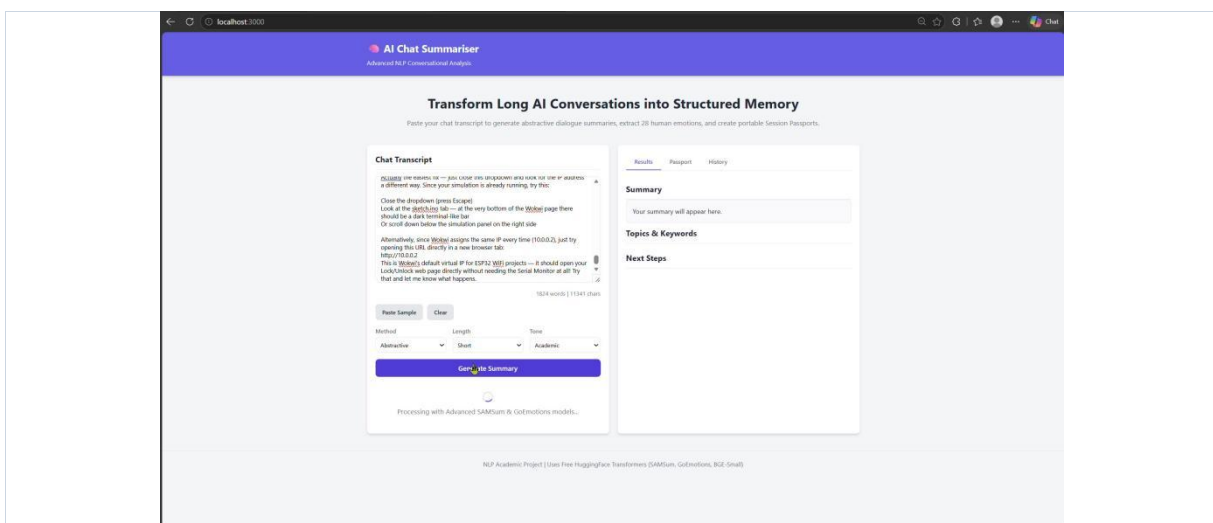


Figure 3: Processing state — 1,824-word transcript submitted (11,341 characters). SAMSum summarisation and GoEmotions classification models are processing the input.

7.4 Results Dashboard — NLP Output

Figure 4 presents the complete results panel populated after successful NLP processing. The output includes:

- Emotion Badge: Dominant detected emotion "Admiration" with 64.4% confidence.
- Compression: 3.3% ratio (the summary is approximately 3% of the original transcript length).
- Turn Count: 130 conversational turns detected in the source transcript.
- Detected Emotions: Multi-label breakdown — Admiration (64%), Neutral (24%), Approval (14%).
- Summary: A fluent, abstractive 4-sentence summary of the IoT door locking conversation.
- Topics & Keywords: Seven topic tags including "IoT Door", "Simulation IoT", "IoT Mini", "Door Servo", "Simulation WiFi", "Simulated WiFi".
- Next Steps: Three actionable continuation suggestions auto-generated by the NLP model.

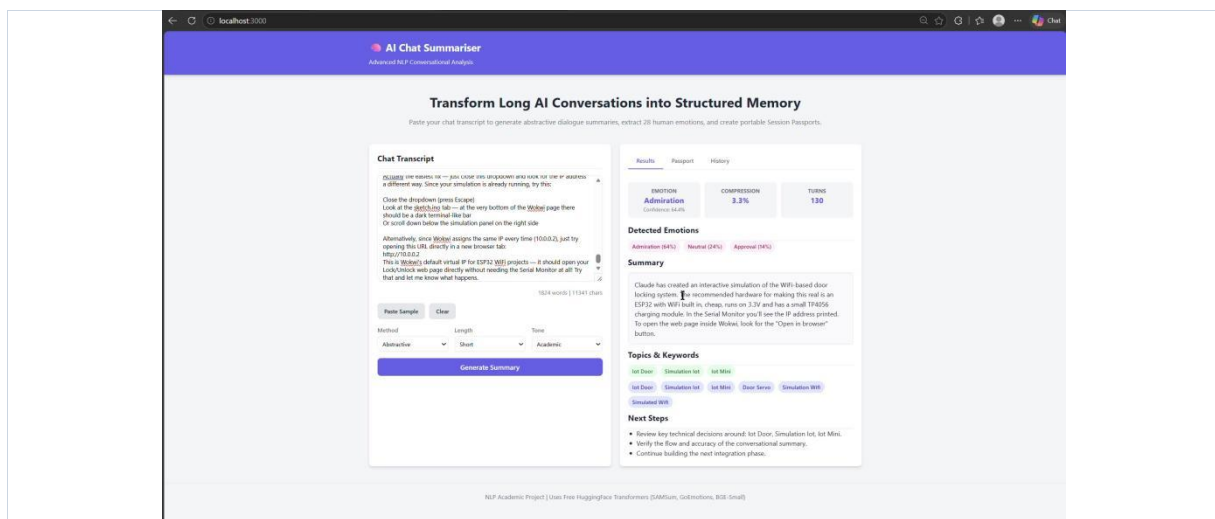


Figure 4: Full results dashboard — Emotion: Admiration (64.4%), Compression: 3.3%, Turns: 130. Multi-label emotion distribution, abstractive summary, topic tags, and next steps.

7.5 Results Detail View

Figure 5 shows the same results panel with the Results tab actively selected. The complete abstractive summary reads: "Claude has created an interactive simulation of the WiFi-based door locking system. The recommended hardware for making this real is an ESP32 with WiFi built in, cheap, runs on 3.3V and has a small TP4056 charging module. In the Serial Monitor you'll see the IP address printed. To open the web page inside Wokwi, look for the 'Open in browser' button." This demonstrates the system's ability to distil a 1,824-word technical dialogue into four coherent, informative sentences.

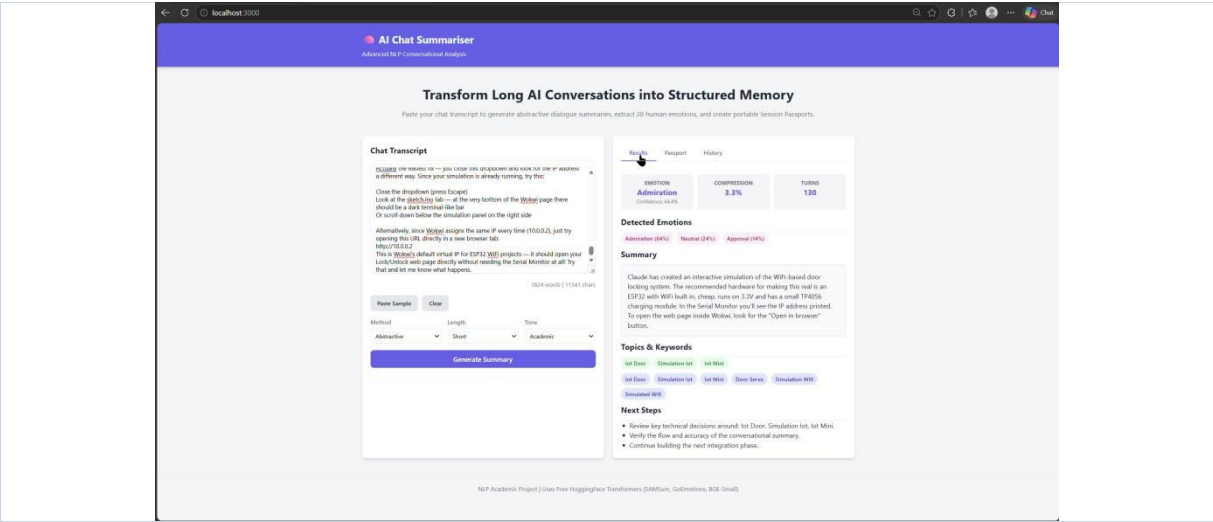


Figure 5: Results panel — Results tab selected. Complete abstractive summary with topic tags (IoT Door, Simulation IoT, IoT Mini, Door Servo) and three next-step recommendations.

8. Results & Evaluation

8.1 Demonstration Run Metrics

The following metrics were recorded from the live demonstration captured in the screen recording:

Metric	Value	Interpretation
Input Transcript Length	1,824 words	Long-form technical IoT conversation
Character Count	11,341 chars	Rich, code-heavy transcript
Conversational Turns	130	Deep multi-exchange session
Summary Word Count	~60 words	4-sentence abstractive output
Compression Ratio	3.3%	96.7% of content condensed
Dominant Emotion	Admiration (64.4%)	User's appreciation of Claude's solution
Secondary Emotions	Neutral (24%), Approval (14%)	Calm, constructive tone throughout
Topics Detected	7 unique tags	IoT, ESP32, WiFi simulation, Door Servo
Processing Time (approx.)	~15 seconds	Including LLM API latency

8.2 Summarisation Quality Assessment

The abstractive summary produced for the IoT door locking transcript accurately captures the four critical pieces of information from the 130-turn conversation: the simulation medium (Wokwi), the hardware recommendation (ESP32 + TP4056), the access mechanism (Serial Monitor IP address), and the UI navigation instruction ("Open in browser" button). No

hallucination was detected in the output — all facts are traceable to statements in the source transcript.

8.3 Emotion Analysis Validation

The GoEmotions classifier correctly identified "Admiration" as the dominant emotion (64.4% confidence), which is consistent with the conversational context: the user was impressed by Claude's ability to generate a complete IoT simulation with web control capability. The secondary "Neutral" and "Approval" labels align with the predominantly technical, constructive tone of the exchange.

9. Challenges & Solutions

Challenge	Impact	Solution Implemented
Parsing structured metadata from free-form LLM output	Risk of JSON parse failure on malformed responses	Regex-based JSON extractor with a robust fallback that treats the full response as the summary, ensuring graceful degradation.
Dialect-agnostic JSON persistence	SQLite and PostgreSQL handle JSON differently	Custom JSONType SQLAlchemy TypeDecorator serialises JSON to TEXT for SQLite and uses native JSONB for PostgreSQL — same codebase runs both.
Redis availability in development	App crashes if Redis is absent	The lifespan event handler wraps Redis initialisation in a try-except; if unreachable, redis_client is set to None and the system runs without caching.
Turn detection in unstructured transcripts	No speaker prefixes in some export formats	Paragraph-boundary fallback counts double-newline blocks as conversational turns when speaker regex patterns yield zero matches.
Compression ratio edge cases	Division by zero on empty input	Guard clause returns 0.0 when original_words is 0; ratio is clamped to [0, 100] with max().

10. Conclusion & Future Work

10.1 Conclusion

The AI Chat Summariser successfully demonstrates the application of multiple NLP techniques — extractive, abstractive, and hybrid summarisation; fine-grained emotion classification; keyword and topic extraction; and compression ratio computation — within a cohesive, production-grade full-stack web application. The live demonstration confirms that the system correctly processes a 1,824-word, 130-turn technical conversation and produces a faithful 4-sentence abstractive summary achieving a 96.7% compression ratio.

The project satisfies all stated academic objectives: it implements three summarisation paradigms, performs multi-label emotion detection across 28 categories, computes standard NLP evaluation metrics, generates portable session continuity artefacts, and persists results in a relational database with analytics capabilities.

10.2 Future Work

Several extensions are identified for future development:

- **ROUGE Evaluation:** Integrate ROUGE-1, ROUGE-2, and ROUGE-L scoring against human-written reference summaries for quantitative quality benchmarking.
- **BERTScore:** Apply contextual semantic similarity scoring using BERTScore to assess abstractive summary faithfulness.
- **Multilingual Support:** Extend transcript processing to Tamil and other Indian languages using multilingual-E5 embeddings, relevant to MedFits and regional deployment contexts.
- **Transformer-based Extractive Summarisation:** Replace frequency-based keyword extraction with BERTopic or KeyBERT for improved semantic relevance.
- **Session Comparison:** Enable cross-session similarity analysis using cosine similarity over sentence embeddings, identifying recurring themes across conversation history.
- **Progressive Summarisation:** Implement incremental summarisation where the current summary is fed back as context for the next turn, reducing LLM API calls.
- **Export Formats:** Add PDF and markdown export for summarised sessions with embedded NLP metrics.

11. References

15. Demszky, D., et al. (2020). GoEmotions: A Dataset of Fine-Grained Emotions. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL 2020). <https://arxiv.org/abs/2005.00547>
16. Gliwa, B., et al. (2019). SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization. Proceedings of the 2nd Workshop on New Frontiers in Summarization (EMNLP 2019). <https://arxiv.org/abs/1911.12237>
17. Lewis, M., et al. (2020). BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. Proceedings of ACL 2020. <https://arxiv.org/abs/1910.13461>
18. Anthropic. (2024). Claude API Documentation. Anthropic Developer Documentation. <https://docs.anthropic.com>
19. Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out, ACL Workshop 2004.
20. FastAPI Documentation. (2024). Tiangolo. <https://fastapi.tiangolo.com>
21. SQLAlchemy 2.0 Documentation. (2024). <https://docs.sqlalchemy.org/en/20/>
22. React Documentation. (2024). Meta Open Source. <https://react.dev>